



CryptoVerse

Project Documentation & Report

A real-time cryptocurrency dashboard built with React, Vite, and Ant Design — live prices, charts, portfolio tracking, and market insights.

Version 1.0 · June 2026

Contents

Part I — Project Report

1. Executive Summary
2. What the Project Is
3. Work Completed
4. Technology Stack
5. Key Metrics — Before vs After
6. Known Limitations
7. Future Roadmap
8. Conclusion

Part II — Technical Documentation

1. Getting Started
2. Project Structure
3. Architecture & Data Flow
4. State Management
5. API Layer
6. Theming System
7. Components Reference
8. Key Feature Logic
9. Routing
10. Environment Variables
11. How to Extend
12. Build & Deploy

CryptoVerse — Project Report

A high-level overview of what the project is, the work carried out, the technology behind it, and where it stands today.

1. Executive Summary

CryptoVerse is a real-time cryptocurrency dashboard built with React. It lets users track live prices and charts for hundreds of coins, read market news, browse exchanges, manage a personal portfolio with live profit/loss, keep a watchlist, and gauge overall market sentiment.

The project began as an ageing Create React App demo with security issues and deprecated dependencies. It has since been **modernised, secured, redesigned, and significantly expanded** into a feature-rich, installable Progressive Web App.

Headline outcomes:

- Security vulnerabilities reduced from **68 to 0**.
 - Build tooling migrated from the deprecated Create React App to **Vite**.
 - Dependency count cut from roughly **1,500 to 524** packages.
 - A complete **dark/light themed UI** with a gold accent identity.
 - Eight major **new user features** (portfolio, watchlist, live prices, and more).
-

2. What the Project Is

CryptoVerse is a single-page application (SPA) that talks directly to public crypto data APIs from the browser. There is no custom backend; user data (portfolio, watchlist, preferences) is stored locally in the browser.

Core sections:

- **Home** — global market stats, Fear & Greed index, top gainers/losers, a preview of the top coins and latest news.
- **Cryptocurrencies** — searchable, sortable list of coins with live prices and trend sparklines.
- **Coin details** — full statistics, an interactive price-history chart, a currency converter, and links.
- **Portfolio** — track holdings and live profit/loss with an allocation chart.
- **Watchlist** — a personal shortlist of starred coins.
- **Exchanges** — a ranked list of crypto exchanges.

- **News** — filterable crypto news feed.
-

3. Work Completed

The project was delivered in four phases.

Phase 1 — Audit & Modernisation

- Performed a deep audit: hardcoded API keys, 68 npm vulnerabilities, and many deprecated/dead dependencies were identified.
- Migrated the build system from **Create React App** to **Vite 8**.
- Upgraded all major libraries: **React 19**, **Ant Design 6**, **React Router 7**, **Redux Toolkit 2**, and replaced `moment` with `dayjs`.
- Removed unused dependencies (`axios`, `newsapi`, `html-react-parser`) and the obsolete service-worker tooling.
- Moved all API keys into a git-ignored `.env` file.

Phase 2 — UI Redesign

- Built a **modern dark-dashboard design system** with a gold/amber accent.
- Added a full **dark/light theme** with a toggle, driven by Ant Design's theme engine and CSS variables.
- Replaced brittle responsive hacks with proper grid/flex layouts that adapt cleanly from desktop to mobile.

Phase 3 — Feature Expansion

- **Portfolio** with live profit/loss and an allocation donut chart.
- **Watchlist** of starred coins.
- **Live prices** that auto-refresh and flash green/red on change.
- **Sparklines** (mini trend charts) on every coin card.
- **Sort & filter** controls and a **Top Gainers / Top Losers** section.
- **Fear & Greed index** and accurate **global market stats** widgets.
- A global **currency selector** (USD, EUR, GBP, BTC, ETH).
- A **price converter** on each coin detail page.

Phase 4 — Navigation Redesign

- Restructured the sidebar: brand on top, navigation in the middle, and the **theme toggle + currency selector pinned to the bottom**.
- Added gradient backgrounds, an active-item highlight, and a polished mobile

drop-down menu.

4. Technology Stack

Area	Technology
Framework	React 19
Build tool	Vite 8
UI library	Ant Design 6
State / data	Redux Toolkit 2 + RTK Query
Routing	React Router 7
Charts	Chart.js 4 + react-chartjs-2
Dates	Day.js
PWA	vite-plugin-pwa (Workbox)
Testing	Vitest (configured)

Data sources: **Coinranking** (via RapidAPI), **CoinGecko**, **NewsAPI**, and **alternative.me** (Fear & Greed).

5. Key Metrics — Before vs After

Metric	Before	After
Security vulnerabilities	68 (2 critical, 32 high)	0
Build tool	Create React App (deprecated)	Vite 8
Installed packages	~1,500	524
Theme support	Light only	Dark + Light toggle
User features	Read-only browsing	Portfolio, watchlist, live data, widgets
API key handling	Hardcoded in source	Environment variables

6. Known Limitations

- **News in production** — NewsAPI's free tier only allows requests from

`localhost`, so the news feed will not load on a deployed site. It fails gracefully with a message. A small server-side proxy or alternative source is required to fix this in production.

- **API key rotation pending** — the original keys were moved to `.env` but not yet revoked; they remain in git history and should be rotated.
 - **Demo data key** — the Coinranking demo key returns a truncated global market cap, so global statistics are sourced from CoinGecko instead.
 - **Bundle size** — the initial JavaScript bundle is ~1.1 MB; route-level code splitting would reduce it.
 - **Automated tests** — Vitest is configured but no test suites have been written yet.
-

7. Future Roadmap

- Server-side proxy (e.g. a serverless function) to fix news and fully hide keys.
 - Route-level code splitting to cut initial load time.
 - Price alerts via the existing PWA service worker (push notifications).
 - A broader visual pass: bento-grid dashboard, richer cards, and motion.
 - A test suite covering state logic and key components.
-

8. Conclusion

CryptoVerse has evolved from an insecure, outdated demo into a modern, secure, and feature-rich crypto dashboard. The foundations — current tooling, a clean theming system, persistent local state, and a flexible API layer — make it straightforward to extend further. The remaining items are well understood and documented, providing a clear path forward.

CryptoVerse — Technical Documentation

Developer reference for the CryptoVerse codebase: setup, structure, architecture, state, APIs, theming, and how to extend the project.

1. Getting Started

Prerequisites: Node.js 20.19+ or 22.12+ and npm.

```
# 1. Install dependencies
npm install

# 2. Configure API keys
cp .env.example .env # then fill in your own keys

# 3. Start the dev server
npm run dev

# Production build / preview
npm run build
npm run preview
```

You need free API keys from RapidAPI (Coinranking) and NewsAPI. The CoinGecko key is optional. All keys live in `.env`, which is git-ignored.

npm scripts:

Script	Action
<code>npm run dev</code>	Start the Vite dev server
<code>npm run build</code>	Production build to <code>dist/</code>
<code>npm run preview</code>	Preview the production build
<code>npm test</code>	Run Vitest

2. Project Structure

```
cryptoverse/
├─ index.html           App entry HTML (Vite)
├─ vite.config.js       Vite + PWA + Vitest config
├─ .env / .env.example  API keys
├─ public/              Static assets (icons, robots.txt)
├─ src/
│   ├─ main.jsx          React entry: Router + Redux + Theme providers
│   ├─ App.jsx           Layout + routes
│   ├─ App.css           Full design system (themes, layout, components)
│   └─ app/
│       ├─ store.js       Redux store + persistence subscriber
│       └─ storage.js     localStorage load/save helpers
│   └─ features/          Redux state slices
│       ├─ watchlistSlice.js
│       ├─ portfolioSlice.js
│       └─ settingsSlice.js
│   └─ services/          RTK Query API definitions
│       ├─ cryptoApi.js   Coinranking (coins, details, history)
│       ├─ cryptoExchange.js CoinGecko (exchanges, global stats)
│       └─ cryptoNewsApi.js NewsAPI (news fetch)
│   └─ theme/ThemeProvider.jsx Dark/light theme + Ant Design config
│   └─ constants/currencies.js Currency UUIDs
│   └─ utils/format.js    Price/number/percent formatting
│   └─ components/        UI components (see section 7)
```

3. Architecture & Data Flow

CryptoVerse is a client-only SPA. The data flow is:

```
External APIs → RTK Query / fetch → Redux store → React components
```

The core mental model:

A user action dispatches to a Redux slice; components re-read state with `useSelector`; data queries re-run when their arguments change; the UI updates. State that must survive a refresh is mirrored to `localStorage`; all visual theming is driven by CSS variables and Ant Design's theme engine.

Provider nesting (in `main.jsx`):

```
Router > Redux Provider > ThemeProvider > App
```


4. State Management

State lives in Redux Toolkit slices under `src/features/`.

Slice	State	Actions
<code>watchlistSlice</code>	<code>ids: []</code>	<code>toggleWatch(uuid)</code>
<code>portfolioSlice</code>	<code>holdings: []</code>	<code>addHolding</code> , <code>removeHolding</code>
<code>settingsSlice</code>	<code>currency</code>	<code>setCurrency</code>

Persistence. `src/app/store.js` subscribes to the store and writes the three user slices to `localStorage` on every change. Each slice reads its initial value back on startup via `loadState`.

```
store.subscribe(() => {
  const state = store.getState();
  saveState("cv-watchlist", state.watchlist.ids);
  saveState("cv-portfolio", state.portfolio.holdings);
  saveState("cv-currency", state.settings.currency);
});
```

localStorage keys: `cv-theme`, `cv-watchlist`, `cv-portfolio`, `cv-currency`.

A holding object has the shape:

```
{ id, uuid, name, symbol, iconUrl, quantity, buyPrice }
```

`buyPrice` is always stored in USD so profit/loss math stays consistent regardless of the selected display currency.

5. API Layer

Three data sources, each defined with RTK Query (or a plain `fetch`).

Coinranking — `services/cryptoApi.js`

Hook	Returns
<code>useGetCryptosQuery(args)</code>	List of coins (+ sparklines)
<code>useGetCryptoDetailsQuery(args)</code>	Full detail for one coin
<code>useGetCryptoHistoryQuery(args)</code>	Price history for the chart

`getCryptos` accepts `{ count, orderBy, orderDirection, referenceCurrencyUuid }`:

- `orderBy` / `orderDirection` drive **sorting** (`marketCap` , `price` , `24hVolume` , `change`). Top Gainers = `change` + `desc` ; Top Losers = `asc` .
- `referenceCurrencyUuid` makes the API return prices in the chosen currency.

Live prices use RTK Query polling:

```
useGetCryptosQuery(args, { pollingInterval: 30000 }); // refetch every 30s
```

CoinGecko — `services/cryptoExchange.js`

Hook	Returns
<code>useGetCryptoExchangeQuery()</code>	Ranked exchanges
<code>useGetGlobalStatsQuery()</code>	Global market cap, BTC dominance, 24h change

Global stats come from CoinGecko because the Coinranking demo key returns a truncated total market cap.

NewsAPI — `services/cryptoNewsApi.js`

`fetchCryptoNews(category, count, searchQuery, sortBy)` calls NewsAPI directly. Note: the free tier only works from `localhost` .

alternative.me

The Fear & Greed index is fetched directly inside `components/FearGreed.jsx` .

6. Theming System

`src/theme/ThemeProvider.jsx` controls both layers of the theme:

1. **Ant Design components** — a `ConfigProvider` swaps `darkAlgorithm` / `defaultAlgorithm` and sets `colorPrimary` to the gold accent `#f7a600` .

1. **Custom CSS** — a `data-theme="dark" | "light"` attribute on the `<html>` element flips CSS variables defined in `App.css` (`--bg` , `--text` , `--card-bg` , and so on).

Usage:

```
import { useTheme } from "../theme/ThemeProvider";
const { mode, toggleTheme } = useTheme();
```

The chosen mode persists to `localStorage` under `cv-theme` .

7. Components Reference

Component	Purpose
<code>Navbar</code>	Sidebar: brand, navigation, currency + theme controls
<code>HomePage</code>	Composes global stats, widgets, top coins, news
<code>Cryptocurrencies</code>	Coin grid with search/sort; also serves watchlist mode
<code>CryptoDetails</code>	Single coin: stats, chart, converter, links
<code>Portfolio</code>	Holdings, live P&L, allocation donut
<code>Exchanges</code>	Ranked exchange list
<code>News</code>	Filterable news feed
<code>LineChart</code>	Price-history chart (theme-aware)
<code>Sparkline</code>	Inline-SVG mini trend chart
<code>AnimatedPrice</code>	Price that flashes green/red on change
<code>WatchButton</code>	Star toggle for the watchlist
<code>Converter</code>	Coin-to-fiat calculator
<code>GlobalStats</code>	Global market stat tiles
<code>FearGreed</code>	Fear & Greed gauge
<code>MarketMovers</code>	Top gainers / losers lists
<code>Loader</code>	Loading spinner

8. Key Feature Logic

Sparkline (`Sparkline.jsx`) — draws a hand-built SVG `<path>` from the 24 price points the API returns, normalised with `(price - min) / range`. Using SVG instead of a Chart.js canvas keeps 100 cards fast.

AnimatedPrice (`AnimatedPrice.jsx`) — a `useRef` stores the previous price; a `useEffect` compares old vs new on each render and flashes green (up) or red (down) for ~900 ms.

WatchButton (`WatchButton.jsx`) — cards are wrapped in a `<Link>`, so the star calls `preventDefault()` and `stopPropagation()` to avoid navigating when toggled.

Portfolio P&L (`Portfolio.jsx`) — for each holding:

```
value = currentPrice * quantity
cost  = buyPrice * quantity
pnl   = value - cost
pnl%  = pnl / cost * 100
```

The portfolio always values in USD (`USD_UUID`) so buy-price math is unaffected by the display currency.

Currency threading — the navbar dispatches `setCurrency` ; every data-driven component reads `useSelector(selectCurrency)` and passes `currency.uuid` into its query. RTK Query treats the new argument as a new request and refetches, so one dropdown re-prices the whole app.

9. Routing

Defined in `App.jsx` with React Router 7:

Path	Component
<code>/</code>	HomePage
<code>/cryptocurrencies</code>	Cryptocurrencies
<code>/watchlist</code>	Cryptocurrencies (watchlist mode)
<code>/portfolio</code>	Portfolio
<code>/coin/:uuid</code>	CryptoDetails
<code>/exchanges</code>	Exchanges
<code>/news</code>	News

10. Environment Variables

All client variables must be prefixed with `VITE_` to be exposed to the app.

Variable	Purpose
<code>VITE_RAPIDAPI_KEY</code>	Coinranking key (RapidAPI)
<code>VITE_RAPIDAPI_HOST</code>	Coinranking host
<code>VITE_NEWS_API_KEY</code>	NewsAPI key
<code>VITE_COINGECKO_API_KEY</code>	CoinGecko key (optional)

Note: in a pure client app, these values ship in the built JavaScript and are not truly secret. To keep keys private, proxy the requests through a backend.

11. How to Extend

Add a new page:

1. Create a component in `src/components/` and export it from `src/components/index.js`.
1. Add a `<Route>` in `App.jsx`.
2. Add an entry to `navItems` in `Navbar.jsx`.

Add a new piece of persistent state:

1. Create a slice in `src/features/`.
2. Register its reducer in `src/app/store.js` and persist it in the subscriber.
3. Read it anywhere with `useSelector`.

Add a new API endpoint:

1. Add an `endpoints` entry to the relevant file in `src/services/`.
2. Export the generated hook and use it in a component.

12. Build & Deploy

- `npm run build` outputs a static site to `dist/`.
- Any static host (Vercel, Netlify, GitHub Pages) can serve it.
- Set the `VITE_*` environment variables in the host's dashboard.
- The app is an installable PWA; the service worker is generated at build time by `vite-plugin-pwa`.